ECE 122: Introduction to Programming for ECE - Spring 2026

Project 1: Mystery Deduction Game

A Basic Procedural Programming Example

Due Date: March 13, 2026
Upload Submissions to Gradescope

Objectives:
This project is designed to reinforce and assess the core learning objectives of ECE 122 as defined in the course syllabus. By completing the Mystery Deduction Game project, students will:

- Program using Python fundamentals (variables, input/output, functions)
- Apply control flow structures (if/elif/else, for loops, while loops)
- Use lists to store and manage structured data
- Translate logical deduction into a step-by-step algorithm
- Develop modular, well-structured procedural programs

Project Description:

The goal of this project is to design and implement a text-based mystery deduction game inspired by the board game Clue.
In the original game, one suspect, one location, and one item are secretly chosen, and players use logical elimination to determine the correct combination.
In this programming version (single-player):

- The computer randomly selects and hides one suspect, one location, and one item.
- The player repeatedly makes guesses.
- The program reports how many parts are correct (0–3), without revealing which ones.
- A deduction notebook tracks and eliminates incorrect options.
- The player has limited attempts, with penalties for duplicate guesses.
- Optional clue tokens may eliminate one incorrect option.
- The player wins by identifying all three correctly before attempts run out.

This project must be implemented using procedural programming only, and **may use only material covered through Lecture 2.6.**

Project Files:

main.py	**DO NOT EDIT THIS FILE.** This file will display the game prompts and expect integer input from the user. The options provided will correspond to functions that you will write in mystery_game.py. When you run your program, run this file.
mystery_game.py	Implement the required functions mentioned under Function Overview.

Work Distribution:

To ensure equal contribution, the following function split must be followed:

Student 1:

- build_game()
- use_clue_token()
- display_game_state()
- best_clue_category()

Student 2:

- check_and_record_guess()
- update_all_notebooks()
- evaluate_guess()
- is_win()

Each student is responsible for implementing and testing the functions assigned to them.
Both students are expected to understand the full program structure and how all functions interact in main.py.

Function Overview:

Your mystery deduction game should support the following operations:

Build Game Data **Function name: build_game()**	Create and return the starting game data. **Inputs:** ● None **Game elements (must be included):** ● Suspects: Colonel Mustard, Miss Scarlet, Professor Plum, Mrs. Peacock, Mr. Green, Dr. Orchid, Sergeant Gray ● Locations: Library, Kitchen, Ballroom, Study, Conservatory ● Items: Candlestick, Rope, Wrench, Knife, Revolver ● Maximum attempts: 4 **Starter code (already provided):** ● The suspects, locations, and items are shuffled each run to change display order. ● Clue tokens are initialized to a random value of 1 or 2. **Return:** ● The suspects, locations, items, max attempts, and clue tokens
Display Game State **Function name: display_game_state()**	Display the current deduction notebook and remaining resources. This function is called inside the main game loop in main.py before the player selects an action. **Inputs (all values are created and managed in main.py):** ● suspects: Suspect data returned by build_game(). ● locations: Location data returned by build_game(). ● items: Item data returned by build_game(). ● pos_suspect: It tracks which suspects are still possible. ● pos_location: It tracks which locations are still possible. ● pos_item:It tracks which items are still possible. ● attempts:The number of remaining guesses. ● tokens: The number of remaining clue tokens. **Output:** ● None ● Prints the current game state to the console. **How the parameters are used in main.py:** ● All values are passed into this function at the start of each loop iteration. ● The tracking data reflects any updates made by other functions. ● This function is responsible only for displaying the current state. **Rules:**

<table>
<tr>
<td></td>
<td>
<ul>
<li>Must not create or redefine game data.</li>
<li>Must not modify any notebook structures.</li>
<li>Responsible only for displaying the current state.</li>
</ul>
<pre>
ECE 122: Mystery Deduction Game

DEDUCTION NOTEBOOK
Remaining Attempts: 4
Clue Tokens: 1

SUSPECTS:
  1 Mrs. Peacock
  2 Mr. Green
  3 Miss Scarlet
  4 Dr. Orchid
  5 Colonel Mustard
  6 Sergeant Gray
  7 Professor Plum

LOCATIONS:
  1 Kitchen
  2 Ballroom
  3 Conservatory
  4 Study
  5 Library

ITEMS:
  1 Revolver
  2 Wrench
  3 Knife
  4 Candlestick
  5 Rope
</pre>
</td>
</tr>
<tr>
<td>Make a Suggestion

Function name: evaluate_guess()</td>
<td>
Determine which parts of the player’s suggestion match the hidden solution.
Inputs (both values are created and passed in by main.py):
<ul>
<li>guess_idx: The player’s current suggestion.</li>
<li>solution_idx: The hidden solution generated in main.py.</li>
</ul>
Output:
<ul>
<li>A collection of exactly three values indicating whether each category is correct.</li>
<li>The order must correspond to: suspect, location, item.</li>
</ul>
How the parameters are used in main.py:
<ul>
<li>guess_idx is formed from the player’s selected options.</li>
<li>solution_idx is stored internally in main.py.</li>
<li>The returned result is used to:
<ul>
<li>Count how many categories are correct.</li>
</ul></li>
</ul>
</td>
</tr>
</table>

	○ Determine whether the player has won.
	○ Update the deduction notebook
	Rules:
	● Must return exactly three values.
	● The order must remain consistent across all calls.
	● Must not print anything.
	● Must not modify any game data.
	● Must only compare the provided inputs.
	
	Example:
	`Your suggestion:`
	`  It was Mrs. Peacock in the Kitchen with the Revolver`
	`Result: 1 out of 3 correct.`

Check for Win **Function name: is_win()**	Determine whether the player has correctly identified all parts of the mystery. **Input (value is passed in from main.py after calling evaluate_guess()):** ● result_list: The correctness result produced by evaluate_guess() for the current suggestion. **Output:** ● A single value indicating whether the mystery has been fully solved. **How the parameter is used in main.py:** ● result_list is passed directly from evaluate_guess(). ● If this function returns a winning result, main.py ends the game. ● If not, the game continues. **Rules:** ● Must return a single value indicating win or not. ● Must not print anything. ● Must not modify any game data. ● Must only evaluate the provided input. `You solved the mystery! Congrats!`

Update the Deduction Notebook **Function name: update_all_notebooks()**	Update the player's deduction notebook using correctness feedback from the most recent suggestion. This version uses only correctness feedback and does not include witness reveals. **Inputs (all values are created and managed in main.py):** ● pos_suspect: The current suspect tracking data stored in main.py. ● pos_location: The current location tracking data stored in main.py. ● pos_item: The current item tracking data stored in main.py. ● guess_idx: The player's most recent suggestion collected in main.py. ● result_list: The correctness feedback returned by evaluate_guess() in main.py. **Output:** ● None **How the parameters are used in main.py:** ● guess_idx is generated from the player's input. ● result_list is produced by calling evaluate_guess(). ● The tracking data (pos_suspect, pos_location, pos_item) is stored in main.py and passed into this function to be updated directly. **Rules:** ● If a category is incorrect, the guessed option in that category must be eliminated. ● If a category is correct, it must remain possible. ● Only options from the current suggestion may be updated. ● Must modify the provided tracking data directly. ● Must not print anything. **BEFORE:**

```
DEDUCTION NOTEBOOK
Remaining Attempts: 4
Clue Tokens: 1

SUSPECTS:
  1 Dr. Orchid
  2 Mr. Green
  3 Sergeant Gray
  4 Professor Plum
  5 Miss Scarlet
  6 Mrs. Peacock
  7 Colonel Mustard

LOCATIONS:
  1 Study
  2 Conservatory
  3 Ballroom
  4 Kitchen
  5 Library

ITEMS:
  1 Wrench
  2 Revolver
  3 Rope
  4 Knife
  5 Candlestick

Hint: SUSPECTS has the most possibilities left.

Actions:
  1) Make a suggestion
  2) Use a clue token
  3) Quit
Choose 1-3: 1
Choose suspect (1-7): 1
Choose location (1-5): 1
Choose item (1-5): 1

Your suggestion:
  It was Dr. Orchid in the Study with the Wrench
Result: 1 out of 3 correct.
Duplicate penalty: 1 (attempts reduced by this amount)
```

AFTER UPDATING:

```
DEDUCTION NOTEBOOK
Remaining Attempts: 3
Clue Tokens: 1

SUSPECTS:
  1 Dr. Orchid
  2 Mr. Green
  3 Sergeant Gray
  4 Professor Plum
  5 Miss Scarlet
  6 Mrs. Peacock
  7 Colonel Mustard

LOCATIONS:
  2 Conservatory
  3 Ballroom
  4 Kitchen
  5 Library

ITEMS:
  2 Revolver
  3 Rope
  4 Knife
  5 Candlestick

Hint: SUSPECTS has the most possibilities left.

Actions:
  1) Make a suggestion
  2) Use a clue token
  3) Quit
Choose 1-3:
```

Record Guess and Apply Duplicate Penalty
 	
Function name: check_and_record_guess()
	Determine whether the player's current suggestion has already been made and return the appropriate attempt penalty. The guess must then be recorded.
	Inputs (both values are created and managed in main.py):
	● guess_history: The stored record of all previous suggestions made during the game.
	● guess_idx: The player's current suggestion collected from user input in main.py.
	Output:
	● A value indicating how many attempts should be deducted for this guess.
	How the parameters are used in main.py:
	● guess_idx is created from the player's selected options.
	● guess_history is maintained in main.py and passed into this function each time a suggestion is made.
	● The returned value is subtracted from the remaining attempts.
	● The updated history is retained for future duplicate checks.
	Rules:
	● Must determine whether the current suggestion has already been made.
	● Must record the current suggestion in the history.
	● A new suggestion must result in a smaller penalty.
	● A repeated suggestion must result in a larger penalty.
	● Must update the provided history directly.
	● Must not print anything.
	`Choose 1-3: 1`
	`Choose suspect (1-7): 1`
	`Choose location (1-5): 1`
	`Choose item (1-5): 1`
	
	`Your suggestion:`
	`  It was Dr. Orchid in the Conservatory with the Wrench`
	`Result: 0 out of 3 correct.`
	`Duplicate penalty: 1 (attempts reduced by this amount)`
	
	`Choose 1-3: 1`
	`Choose suspect (1-7): 1`
	`Choose location (1-5): 1`
	`Choose item (1-5): 1`
	
	`Your suggestion:`
	`  It was Dr. Orchid in the Conservatory with the Wrench`
	`Result: 0 out of 3 correct.`
	`Duplicate penalty: 2 (attempts reduced by this amount)`

<table>
<tr>
<td>Use a Clue Token

Function name:
use_clue_token()</td>
<td>Use a clue token to eliminate one incorrect option from a selected category.
Inputs (both values are created and managed in main.py):
● possible_list: The current tracking data for a single category (suspects, locations, or items), passed in from main.py.
● solution_index: The correct solution value for that category, stored in main.py.
Output:
● A value indicating whether an option was successfully eliminated.
How the parameters are used in main.py:
● The selected category’s tracking data is passed into this function.
● The corresponding solution value is also passed in.
● If this function reports that an elimination occurred, main.py decreases the number of remaining clue tokens.
● The updated tracking data is used in future calls to display_game_state().
Rules:
● At most one incorrect option may be eliminated per call.
● The correct solution must never be eliminated.
● Only options that are still marked as possible may be eliminated.
● If no eligible option exists, the function must report that no elimination occurred.
● Must update the provided tracking data directly.
● Must not print anything.

<pre>Actions:
  1) Make a suggestion
  2) Use a clue token
  3) Quit
Choose 1-3: 2

Use a clue token on:
  0) Suspects
  1) Locations
  2) Items
  3) Use the suggested category
Choose 0-3: 1
A clue token eliminated one incorrect option.
Clue tokens remaining: 0
Remaining possibilities: Suspects = 7 | Locations = 4 | Items = 5</pre></td>
</tr>
<tr>
<td>Suggest Best Clue Token Category

Function name:</td>
<td>Determine which category currently has the most remaining possible options.
Inputs (all values are created and managed in main.py):
● pos_suspect: The current tracking data for suspects.</td>
</tr>
</table>

best_clue_category()	● pos_location: The current tracking data for locations.
	● pos_item: The current tracking data for items.
	Output:
	● A value indicating which category should receive a clue token:
	○ 0 represents suspects
	○ 1 represents locations
	○ 2 represents items
	How the parameters are used in main.py:
	● The tracking data for all three categories is passed into this function.
	● The returned category value is used to suggest where a clue token may be most effective.
	● If the player selects the suggested option, main.py applies the clue token to that category.
	Rules:
	● Must determine which category currently has the greatest number of remaining possibilities.
	● If there is a tie, priority must follow this order: suspects → locations → items
	● Must not print anything.
	● Must not modify any game data.
	`Actions:`
	`  1) Make a suggestion`
	`  2) Use a clue token`
	`  3) Quit`
	`Choose 1-3: 2`
	
	`Use a clue token on:`
	`  0) Suspects`
	`  1) Locations`
	`  2) Items`
	`  3) Use the suggested category`
	`Choose 0-3: 3`
	`A clue token eliminated one incorrect option.`
	`Clue tokens remaining: 1`
	`Remaining possibilities: Suspects = 6 | Locations = 5 | Items = 5`

Grading Criteria (100 Points Total):

Understanding Test Cases

Test cases are **prewritten scripts** designed to check if your code works correctly. Instead of manually testing every function, these test cases will run specific scenarios automatically and print whether your implementation in playlist.py is correct.

While each test case must independently pass, they collectively should function as a unit to earn full credits.

Test 1: Guess Evaluation & Win Logic	**Functions tested:** evaluate_guess(), is_win() **What it checks:** ● evaluate_guess() returns a list of 3 booleans (one per category) ● is_win() returns True only when all three are True	15 points
Test 2: Notebook Update After a Guess	**Functions tested:** update_all_notebooks() **What it checks:** ● If a category is incorrect, the guessed option is eliminated (set to False) ● If a category is correct, nothing in that category is eliminated	15 points
Test 3: Duplicate Guess Penalty + Guess History	**Functions tested:** check_and_record_guess() **What it checks:** ● First-time guess penalty is 1 ● Exact duplicate guess penalty is 2 ● Guess history records all guesses (including duplicates)	25 points

Test 4: Clue Token + Suggested Category	**Functions tested:** use_clue_token(), best_clue_category() **What it checks:** ● use_clue_token() eliminates the lowest-index incorrect True option ● If only the correct option remains, it returns False ● best_clue_category() returns the category with most remaining True values	15 points
Test 5: Display Notebook Output	**Functions tested:** display_game_state() **What it checks:** ● Required headings print correctly ● Attempts and tokens print ● Only options with True are shown (options with False do NOT print)	15 points
Criteria 1.0	Structure & Comments Explaining Code.	15 points

Group Requirements:
This is a two-person project. Groups will continue through Project 3, so choose your partner carefully.
Add both names, emails, and SPIRE IDs to this spreadsheet:
https://docs.google.com/spreadsheets/d/1fzPVro5jR_lCfPeobzk29CJQ_WSDOSJkkEBznaQSQkI/edit?usp=sharing
If you do not have a partner, add your name and email and we will assign one.

Submission Guidelines:
Submit your mystery_game.py on Gradescope.
Make sure all required functions are correctly implemented. Only use programming concepts discussed in class (up to Chapter 2.6).
Using AI or copying other students' code is not allowed and will result in grade deductions and possible disciplinary action.

Code should be properly commented and formatted for readability.
This project is designed to be implemented incrementally. You are strongly encouraged to run and test each function as you build it.
If you have any questions, reach out to the course TAs.
Happy coding!